Tarea 1: Diseño de Sistemas Secuenciales

Por Ricardo José Colom Palero

En esta presentación, se plantearán las bases para el desarrollo y realización de la primera práctica de la asignatura, en la que se abordará el diseño de sistemas digitales secuenciales utilizando para ello, lenguajes de descripción hardware, como el Verilog HDL.

Previamente a la realización de la tarea debe realizarse el siguiente test sobre cuestiones de sintaxis de Verilog-HDL, para evaluar los conocimientos previos que se poseen.

Task 1: Design of Sequential Systems

By Ricardo José Colom Palero

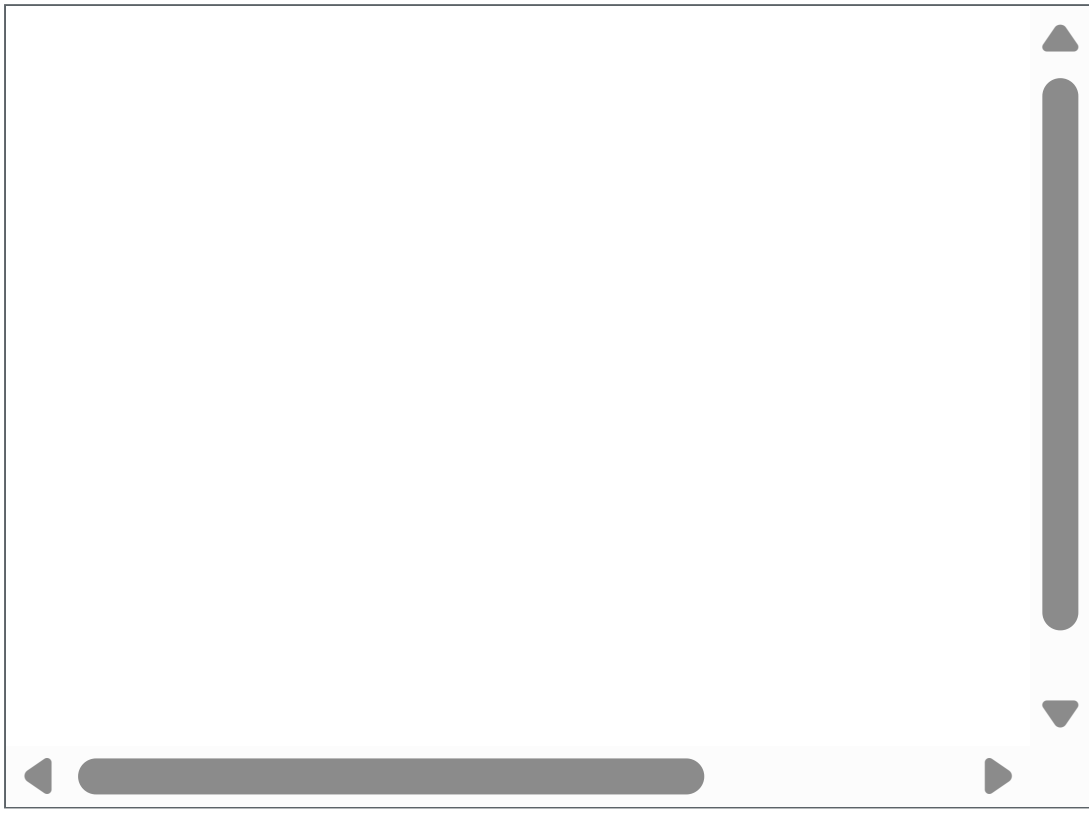
In this presentation, the bases for the development and realization of the first practice of the subject will be considered, in which the design of sequential digital systems will be addressed using hardware description languages, such as the Verilog HDL.

Before the completion of the task, the following test should be done on questions of Verilog-HDL syntax, to evaluate the previous knowledge that is possessed.

Objetivos

Goals

ABRIR EN UNA VENTANA NUEVA



Como objetivos fundamental de esta tarea se plantea el aprendizaje de los conceptos básicos del lenguaje Verilog HDL y su uso en la realización de diseños, incidiendo de manera particular en el uso de bloques procedurales gobernados por el reloj para la modernización de circuitos secuenciales.

Se realizará el diseño de elementos digitales utilizando el lenguaje Verilog HDL, avanzando en el aprendizaje y sus de la jerarquía y los parámetros en Verilog HDL. Por último, habrá que destacar la verificación del funcionamiento de los diseños realizados mediante simulaciones, que permitan comprobar de una manera exhaustiva su correcta realización.

The fundamental objectives of this task are the learning of the basic concepts of the Verilog HDL language and its use in the realization of designs, with particular emphasis on the use of procedural blocks governed by the clock for the modernization of sequential circuits.

The design of digital elements will be done using the Verilog HDL language, advancing in the learning and its hierarchy and parameters in Verilog HDL. Finally, we must highlight the verification of the operation of the designs made by simulations, which allow a thorough verification of their correct performance.

Estructura de la Tarea

Structure of the Task

ABRIR EN UNA VENTANA NUEVA



Todo diseño de sistema secuencial va gobernado por una señal de reloj, proveniente de un oscilador externo al dispositivo, en el que se implementa el diseño. En la placa DE2-115 el oscilador que se dispone tiene una frecuencia de 50 MHz, lo cual significa que los cambios de las señales se producirán cada 20 ns. Cuando se realizan diseños para controlar o utilizar elementos de visualización perceptibles por el ojo humano, hay que tener presente que cambios con un tiempo inferior a 50 ms, no son visibles por los humanos. Así que, no se puede utilizar directamente el oscilador de 50 MHz para realizar cambios que deben ser visibles por el ojo humano; por ejemplo, hacer que un LED parpadee.

En está practica se ve cómo solucionar este problema de visualización, manteniendo los criterios de un correcto diseño síncrono en que todos los flip-flops del diseño deben estar gobernados por la misma señal de reloj.

La práctica de diseño de sistemas secuenciales con Verilog HDL está estructurada en cuatro subtareas en las que se tratan los siguientes conceptos:

- Diseño de contadores en Verilog HDL.
- Diseño de registros de desplazamiento.
- Integración de diseños anteriores para la realización del juego de luces del coche fantástico.

Every sequential system design is governed by a clock signal, coming from an oscillator external to the device, in which the design is implemented. On the DE2-115 board the oscillator that is available has a frequency of 50 MHz, which means that the changes of the signals will occur every 20 ns. When designs are made to control or use visualization elements perceptible by the human eye, it must be borne in mind that changes with a time less than 50 ms, are not visible by humans. So, you can not directly use the 50 MHz oscillator to make changes that must be visible by the human eye; for example, make an LED flash.

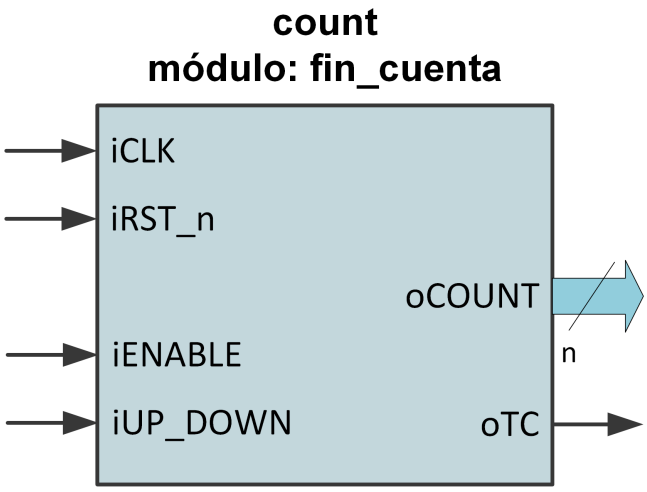
In this task we see how to solve this visualization problem, maintaining the criteria of a correct synchronous design in which all the design flip-flops must be governed by the same clock signal.

The task practice of designing sequential systems with Verilog HDL is structured in four subtasks in which the following concepts are treated:

- Design of counters in Verilog HDL.
- Design of shift registers.
- Integration of previous designs for the realization of the game of knight rider lights.

Subtarea 1: Diseño de un contador parametrizable

Subtask 1: Design a parametric counter



Realizar un contador binario, mediante descripción Verilog HDL que tenga las siguientes características:

- Contador síncrono ascendente - descendente de módulo parametrizable `fin_cuenta`.
- Funcionamiento por flanco de subida del reloj.
- Reset, síncrono (máxima prioridad), activo a nivel bajo.
- Entrada de habilitación de cuenta **enable** que habilita la cuenta.
- Si la entrada **UP_DOWN** está a nivel alto, el contador cuenta hacia arriba. De lo contrario, cuenta hacia abajo.
- Señal de salida **TC** activa cuando el contador llega al fin de cuenta.
- Utilizar la Familia Cyclone IV E de intelFPGA.

Verifique su funcionamiento mediante una simulación que consideré todos los posibles casos de combinaciones de las señales de entrada y realice un barrido completo de las señales de salida.

Design a binary counter, using Verilog HDL, with the following specifications:

- Synchronous increasing/decreasing counter, with parameterizable modulus `fin_cuenta` / `end_count`.
- Operates on the clock's rising edge.
- Synchronous reset (highest priority), active at low level.
- Input signal (**enable**) that enables counter operation. Active high.
- Counts up if the input **UP_DOWN** is high, otherwise it counts down.
- Output signal (**TC, Terminal Count**) that becomes active when the counter reaches the final count number.
- Implement on an IntelFPGA Cyclone IV E device.

Verify its operation with a simulation that considers all possible cases of combinations of input signals and performs a complete monitoring of the output signals.

Un diseñador eficiente debe saber rentabilizar el tiempo que dedica a realizar los diseños. Por ello, es importante saber reutilizar los módulos o bloques anteriormente diseñados. Una cualidad de los lenguajes HDL, que permite incrementar las posibilidades de reutilizar un diseño, es la parametrización de los mismos. Por ejemplo, en muchas ocasiones se requiere el uso de un contador, si previamente se ha realizado ese contador se puede reutilizar. Sin embargo puede ocurrir (y ocurrirá) que el contador realizado sea de 8 bits y el que se necesite de 10 bits. La solución parece sencilla, se toma la descripción del contador de 8 bits y se modifica para hacer uno de 10 bits. Esto tiene el inconveniente de tener que disponer de una colección distinta de archivos con modelizado de contadores para diferentes tamaños.

La mejor solución es disponer de un solo archivo con el modelo del contador, de tal manera que este se haya realizado dependiente de un parámetro de configuración, que se cambiará en función de las necesidades.

En ocasiones puede ser necesario realizar operaciones matemáticas con el parámetro antes de utilizarlo dentro del diseño. Por ejemplo, se quiere a partir de un dato obtener el número de bits necesario para poder representarlo. ¿Cómo se obtiene el número de bits que debe tener el contador, a partir del módulo? La solución es recurrir al uso de una función que permita obtener el número de bits a partir del dato de fin de cuenta o módulo. En la Figura inferior se muestra un ejemplo de cómo hacerlo utilizando la función:

An efficient designer must know how to make the most of the time he dedicates to making the designs. Therefore, it is important to know how to reuse previously designed modules or blocks. A quality of the HDL languages, which allows increasing the possibilities of reusing a design, is the parameterization of them. For example, in many occasions the use of a counter is required, if previously this counter has been made it can be reused. However, it can happen (and will happen) that the counter made is 8 bits and the one that is needed of 10 bits. The solution seems simple, the description of the 8-bit counter is taken and modified to make a 10-bit counter. This has the disadvantage of having to have a different collection of files with modeling of counters for different sizes.

The best solution is to have a single file with the counter model, in such a way that it has been made dependent on a configuration parameter, which will be changed according to the needs.

Sometimes it may be necessary to perform mathematical operations with the parameter before using it within the design. For example, you want from a data to obtain the number of bits necessary to be able to represent it. How do you obtain the number of bits that the counter must have, from the module? The solution is to resort to the use of a function that allows obtaining the number of bits from the end of account or module data. The lower figure shows an example of how to do it using the function: `CLogB2`, which calculates the logarithm in base 2, with

CLogB2, que calcula el logaritmo en base 2, con redondeo al entero más próximo. Tal como se aprecia la función CLogB2, permite obtener el valor de cuantos bits hacen falta para alcanzar el fin de cuenta establecido. La función CLogB2 se encuentra definida en el fichero de funciones matemáticas denominado MathFun.vh, que habrá que incluir previamente en el diseño.

rounding to the nearest integer. As shown by the function CLogB2, it allows to obtain the value of how many bits are needed to reach the end of the established account. The function CLogB2 is defined in the file of mathematical functions called MathFun.vh, which must be previously included in the design.

```
module contador (iCLK, iRST_n, iENABLE, iUP_DOWN, oCOUNT, oTC);

   parameter fin_cuenta = 20;

   `include "MathFun.vh"
   parameter n = CLogB2(fin_cuenta-1);

   input iCLK, iRST_n, iENABLE, iUP_DOWN;
   output reg [n-1:0] oCOUNT;
   output oTC;

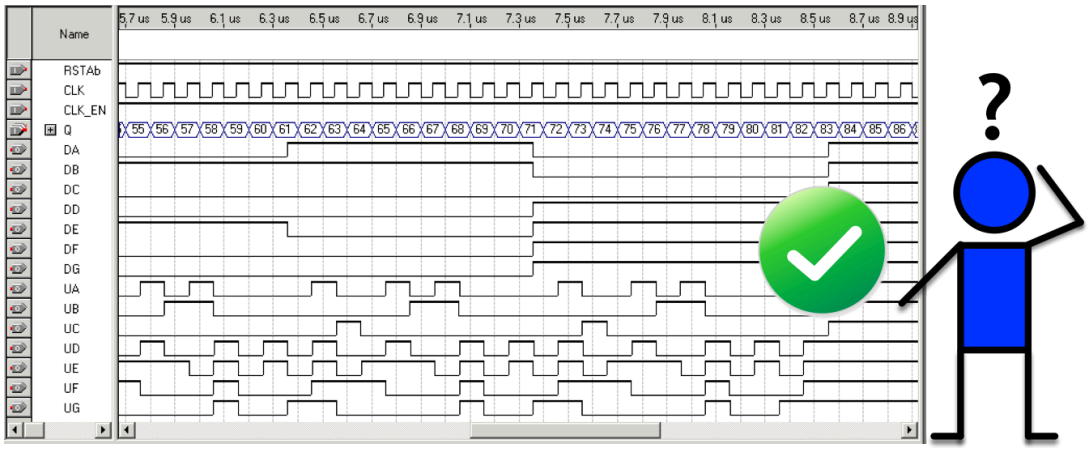
...

endmodule
```

ARCHIVO DE FUNCIONES MATEMÁTICAS: MATHFUN.VH

Verificación del Diseño

Design Verification



Una vez realizado el diseño en Verilog HDL del módulo `count`, hay que verificar su funcionamiento mediante las correspondientes simulaciones. Es necesario comprobar que el contador funciona correctamente y se generan adecuadamente las señales de salida con el valor de la cuenta y el indicador del fin del conteo.

Por ello, es necesario emplear un simulador profesional como es el ModelSim-Altera. No obstante, el simulador ModelSim-Altera necesita de un Testbech para la realización de la simulación. El Testbench contiene la descripción de los estímulos de entrada para la simulación, descritos mediante un lenguaje de descripción hardware como es Verilog HDL.

Como todavía no se tiene experiencia para realizar un Testbench, mediante una descripción textual con Verilog HDL, se deberán seguir los pasos que a continuación se describen:

Once the Verilog HDL design of the `count` module has been carried out, its operation must be verified by means of the corresponding simulations. It is necessary to check that the counter works correctly and the output signals are properly generated with the value of the count and the indicator of the end of the count.

Therefore, it is necessary to use a professional simulator such as the ModelSim-Altera. However, the ModelSim-Altera simulator needs a Testbech to perform the simulation. The Testbench contains the description of the input signals for the simulation, described by a hardware description language such as Verilog HDL.

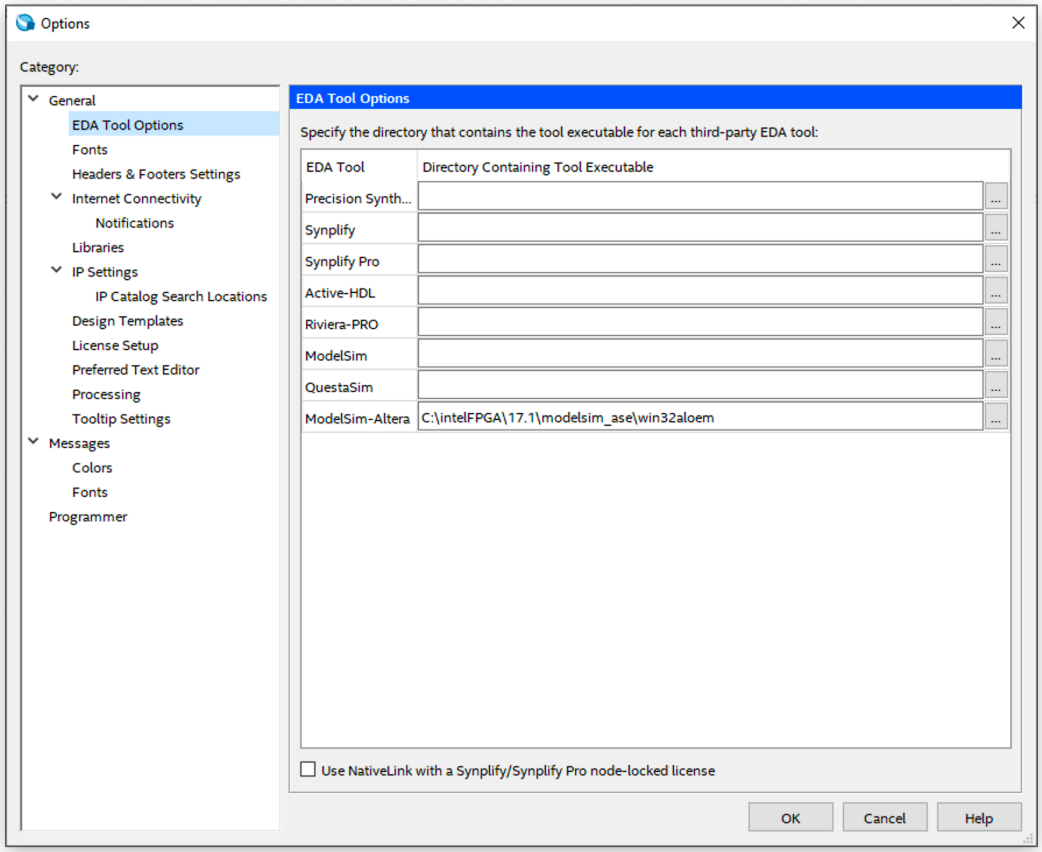
As you still do not have experience to perform a Testbench, by means of a textual description with Verilog HDL, you must follow the steps described below:

Comprobar Configuración Quartus

Check Quartus Configuration

Antes de realizar la simulación conviene comprobar que la herramienta está bien configurada, por lo que hay que verificar que el path de instalación de ModelSim-Altera se encuentra bien introducido en Quartus II. Para ello en el menú **Tools > Options...** se abre la ventana de **Options**. En dicha ventana seleccionando **General > EDA Tool Options** en la columna de **Category**, aparecerá la lista de las herramientas, para ModelSim-Altera hay que configurar el path **C:\intelFPGA\17.1\modelsim_ase\win32aloem** o equivalente.

Before performing the simulation, it is advisable to check that the tool is properly configured, so it is necessary to verify that the installation path of ModelSim-Altera is well introduced in Quartus II. To do this, in the **Tools > Options...** menu, the **Options** window opens. In this window selecting **General > EDA Tool Options** in the **Category** column, the list of tools will appear, for ModelSim-Altera you have to configure the path **C:\intelFPGA\17.1\modelsim_ase\win32aloem** or equivalent.



Creación de los Estímulos con Verilog HDL

La generación de los estímulos para la simulación, se puede realizar mediante el propio Verilog HDL. El procedimiento a seguir, consiste en crear primero el archivo de texto para la descripción de los estímulos. En el menú **File > New**, se abre una ventana en la que se selecciona la opción **Verilog HDL File**. En la nueva pestaña de edición abierta, se escribe el código mostrado a continuación.

Como se trata de un código Verilog para simulación o verificación, su función principal es generar los estímulos, por ello el `module`no dispone de señales de entrada y salida, ya que el código debe generar las entradas para inyectarlas al bloque a verificar. En esta ocasión el bloque a verificar es el `count`, diseñado anteriormente, el cual se instancia dentro del testbench para conectarle las entradas generadas y recoger las salidas.

Stimulus Generation for Verilog HDL

The stimulus generation for the simulation can be done by Verilog HDL itself. The procedure to follow is to first create the text file for the description of the stimulus. In the **File > New** menu, a window opens in which the **Verilog HDL File** option is selected. In the new open edition tab, the code shown below is written.

As it is a Verilog code for simulation or verification, its main function is to generate the stimulus, therefore the module does not have input and output signals, since the code must generate the inputs to inject them into the DUT (Device under Test). This time the block to verify is the `count`, previously designed, which is instantiated inside the testbench to connect the generated inputs and collect the outputs.

```
// Autor: Ricardo José Colom Palero
// Fecha: 10 de diciembre de 2018

`timescale 1ns/1ps

module tb_count ();

localparam T = 20;

reg         CLK, RST_n;
reg         ENABLE, UP_DOWN;
wire   [ 4:0]   COUNT;
wire         TC;

// Instanciación del DUT (Device Under Test)
count #(.fin_cuenta(20)) i1 (   .iCLK(CLK),
                    .iRST_n(RST_n),
                    .iENABLE(ENABLE),
                    .iUP_DOWN(UP_DOWN),
                    .oCOUNT(COUNT),
                    .oTC(TC));

initial
  begin
    RST_n = 1'b0;
    CLK = 1'b0;

    ENABLE = 1'b0;
    UP_DOWN = 1'b0;
    $display("SIMULANDO!!!");
    #(T*2)

    RST_n = 1'b1;

    #(T*2)

    ENABLE = 1'b1;
    UP_DOWN = 1'b1;
    #(T*24)


    $display("Fin Simulacion!!!");
    $stop;

  end

always
  begin
```

```
    #(T/2) CLK <= ~CLK;
  end

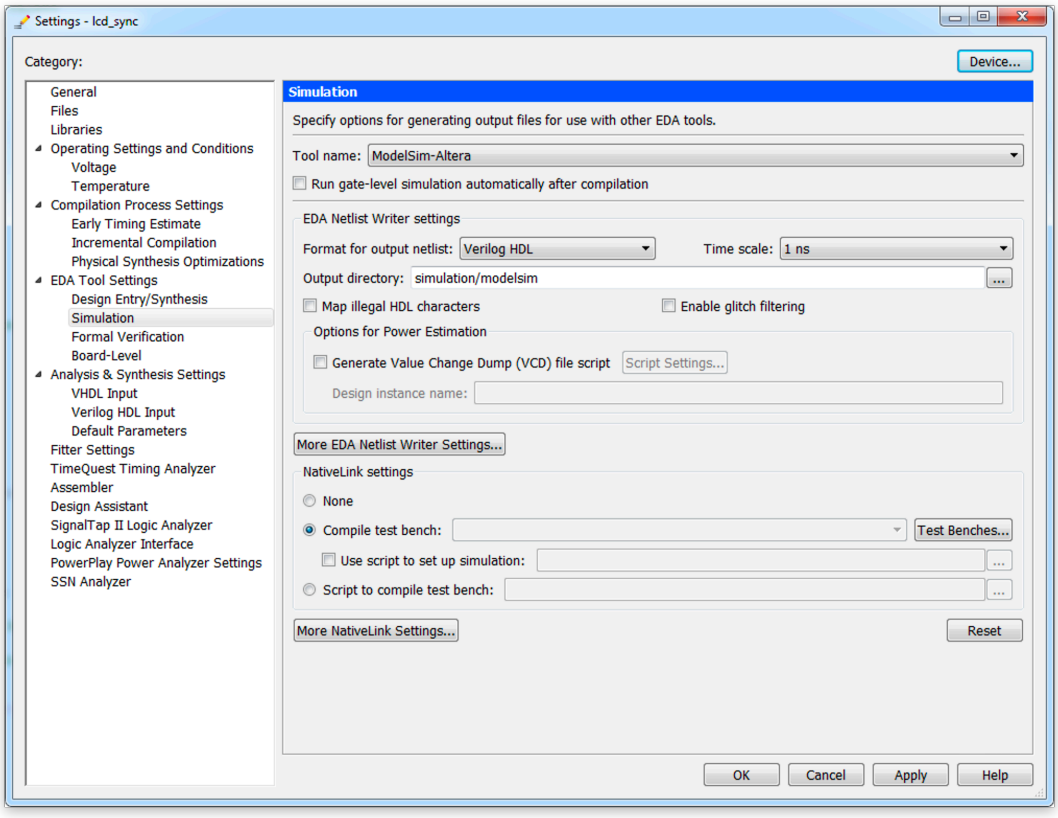
endmodule
```

Una vez escrito el código Verilog HDL, en el menú **File > Save as...**, se guarda con el nombre `tb_count.v`. Este archivo no debe ser compilado con el resto del proyecto, ya que no forma parte del diseño que finalmente se configura en la FPGA, únicamente se utiliza para la verificación.

El siguiente paso es la configuración de la simulación para que encuentre el archivo de estímulos. En el menú: **Assignments > Settings** se abre una nueva ventana, seleccionando la opción de Simulation de la columna izquierda, aparecen las opciones de configuración de la simulación mostradas en la siguiente imagen:

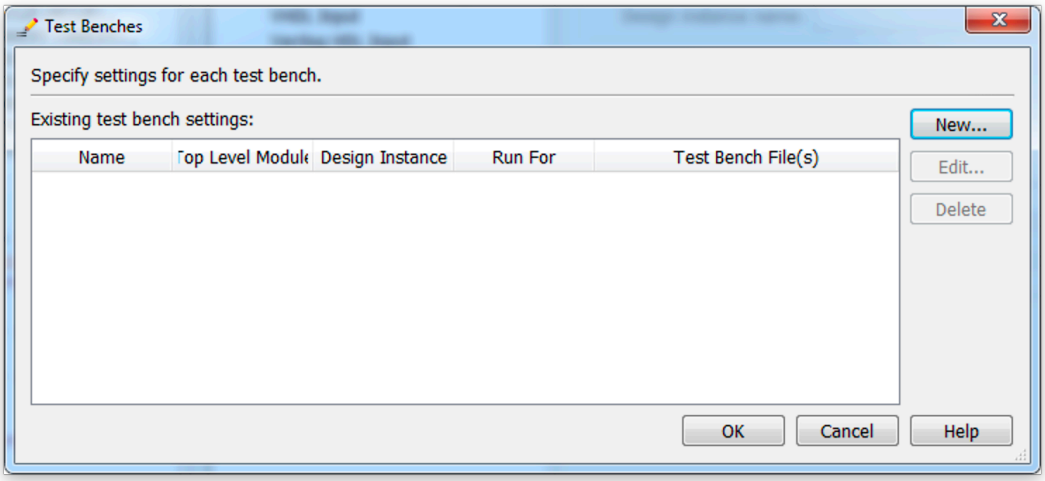
Once the Verilog HDL code is written, in the menu **File > Save as...**, it is saved with the name `tb_count.v`. This file must not be compiled with the rest of the project, since it is not part of the design that is finally configured in the FPGA, it is only used for verification.

The next step is the configuration of the simulation to find the stimulus file. In the menu: **Assignments > Settings** a new window is opened, selecting the Simulation option in the left column, the simulation configuration options shown in the following image appear:



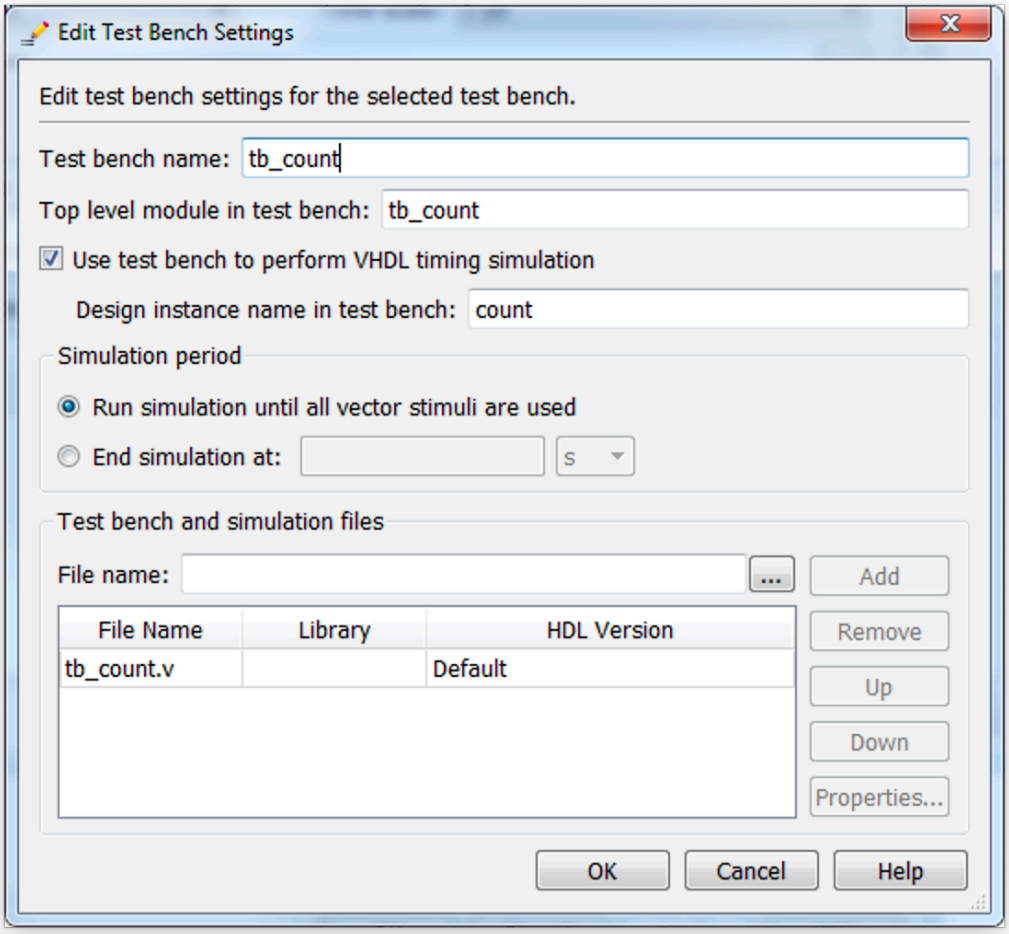
En esta ventana se pulsa el botón **Test Benches...**, que se encuentra por la mitad en el margen derecho. Esto abrirá una nueva ventana...

In this window press the **Test Benches...** button, which is in the middle on the right margin. This will open a new window...



...en la que hay que seleccionar la opción **New**, para añadir un nuevo testbench, configurándolo tal como se muestra en la siguiente imagen:

...in which you have to select the option **New**, to add a new testbench, configuring it as shown in the following image:

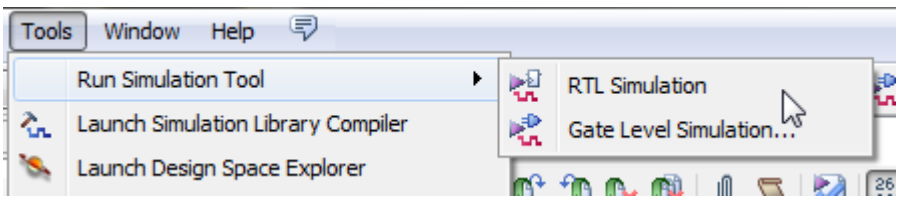


Se retorna mediante los sucesivos **OK**.

It is returned through the successive **OK**.

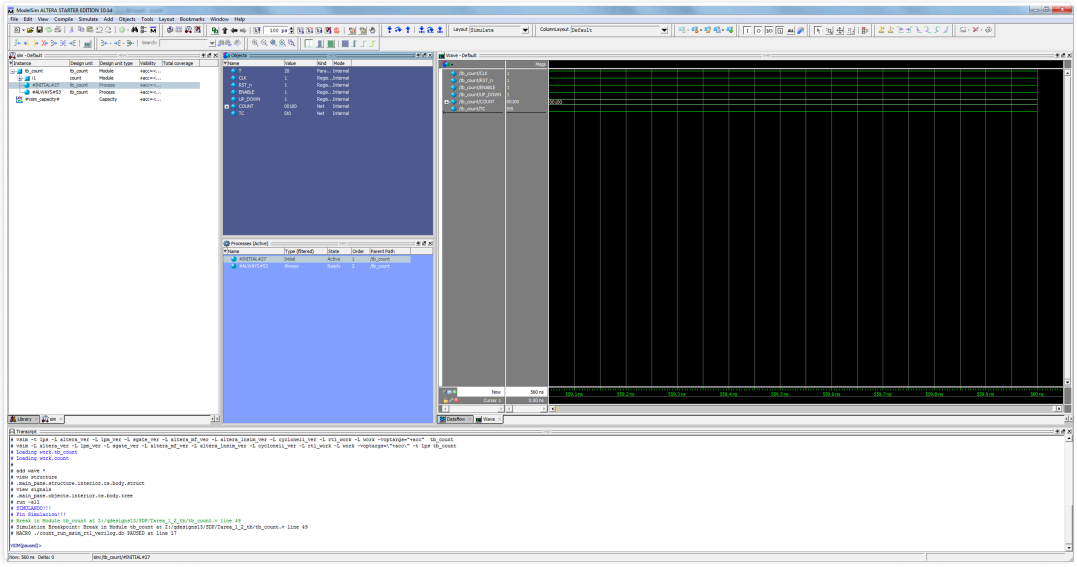
Ahora ya solo resta lanzar la simulación con ModelSim-Altera, para ello en el menú: **Tools > Run Simulation Tools > Run RTL Simulation**, permite realizar una simulación del código RTL previa al proceso de síntesis e implementación.

Now all that remains is to launch the simulation with ModelSim-Altera, for this in the menu: **Tools > Run Simulation Tools > Run RTL Simulation**, allows you to perform a simulation of the RTL code prior to the synthesis and implementation process.



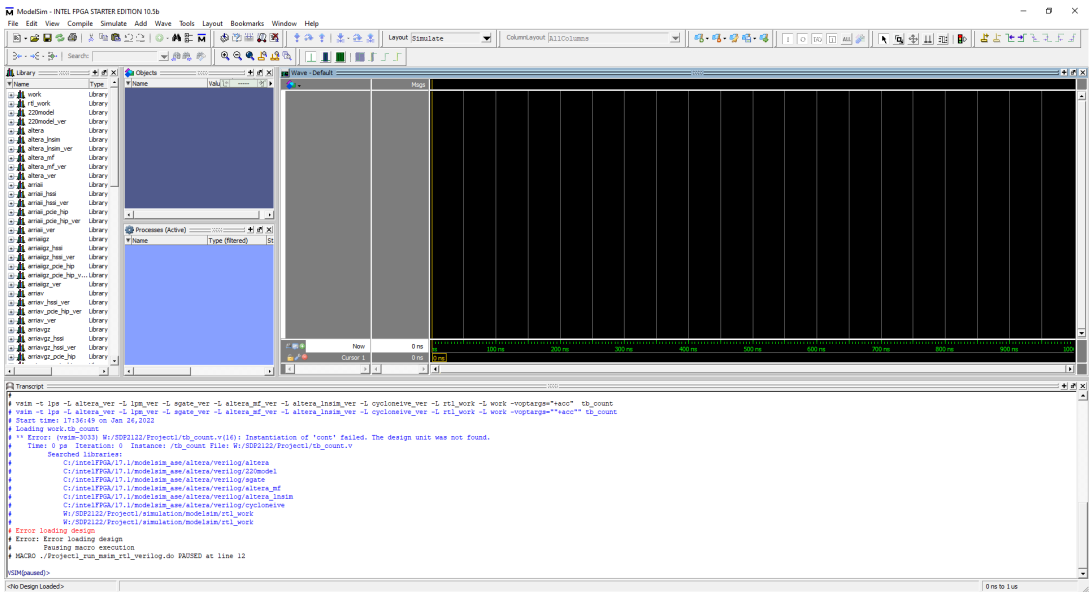
Se abre el simulador ModelSim-Altera, y comienza la simulación según la descripción realizada en el Testbench, mediante el código Verilog. En este caso concreto, la simulación se detendrá cuando transcurra el tiempo indicado, equivalente a alrededor de 20 ciclos del reloj. Así, se puede observar la evolución del contador desde la cuenta inicial 0 a la final 19, para volver a cero.

The ModelSim-Altera simulator is opened and the simulation begins according to the description made in the Testbench, using the Verilog code. In this specific case, the simulation will stop when the indicated time elapses, equivalent to around 20 clock cycles. Thus, the evolution of the counter can be observed from the initial count 0 to the final count 19, to return to zero.



Hay que tener en cuenta que el simulador ModelSim también puede dar errores si el código del testbench no compila correctamente. Debe revisarse siempre la ventana de *Transcript* en la parte inferior de ModelSim para ver si hay errores o warnings. Por ejemplo, en la captura siguiente vemos un error por haber escrito mal el nombre del módulo contador.

Be aware that the ModelSim simulator can give error messages too when the testbench code cannot be compiled correctly! Always check the *Transcript* window at the bottom for errors and warnings. For instance, in the example below we made a typo when typing counter module's name.



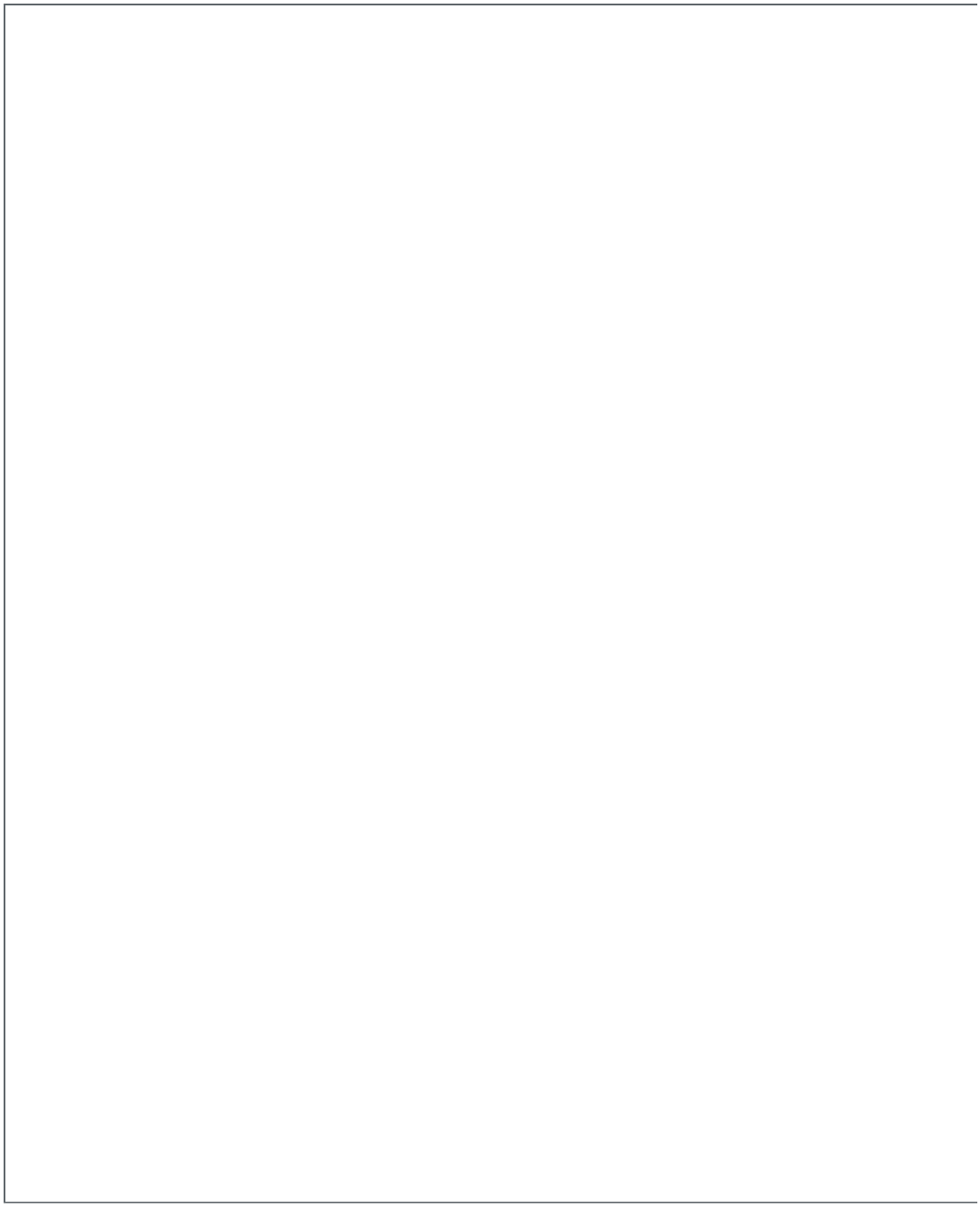
El Simulador ModelSim-Altera

El simulador ModelSim-Altera, está constituido por varias ventanas, en la parte inferior se encuentra la ventana de mensajes, donde el simulador envía diversa información a tener en cuenta. Por ejemplo, si se detectan errores en los archivos verilog HDL del diseño. Por ese motivo es conveniente revisar los mensajes que aparecen, ya que un error en los archivos impide que el proyecto se pueda cargar y por tanto realizar la simulación.

The ModelSim-Altera Simulator

The Simulator ModelSim-Altera, is constituted by several windows, in the lower part is the message window, where the simulator sends various information to take into account. For example, if errors are detected in the verilog HDL files of the design. For this reason, it is convenient to review the messages that appear, since an error in the files prevents the project from being loaded and therefore the simulation can be performed.

ABRIR EN UNA VENTANA NUEVA



En cuanto al resto de ventanas, en el lado izquierdo se encuentra el visor de jerarquía, en él se mostrará la estructura del diseño con todos los archivos que contiene. La ventana que se encuentra en la parte central superior es el visor de señales, muestra las señales del archivo que se encuentra seleccionado en el visor de jerarquía. Estas señales se pueden añadir al visor de ondas que se encuentra en el lado derecho, en él se muestra el resultado de la simulación. Todas las ventanas del ModelSim se pueden desacoplar y visualizarse de manera independiente, lo cual resulta especialmente útil en el caso del visor de ondas, para poder obtener una mejor visualización.

As for the rest of the windows, on the left side there is the hierarchy viewer, it will show the structure of the design with all the files it contains. The window that is in the upper central part is the signal viewer, it shows the signals of the file that is selected in the hierarchy viewer. These signals can be added to the wave viewer on the right side, which shows the result of the simulation. All the windows of the ModelSim can be uncoupled and displayed independently, which is especially useful in the case of the wave viewer, to obtain a better visualization.

ABRIR EN UNA VENTANA NUEVA

Change language

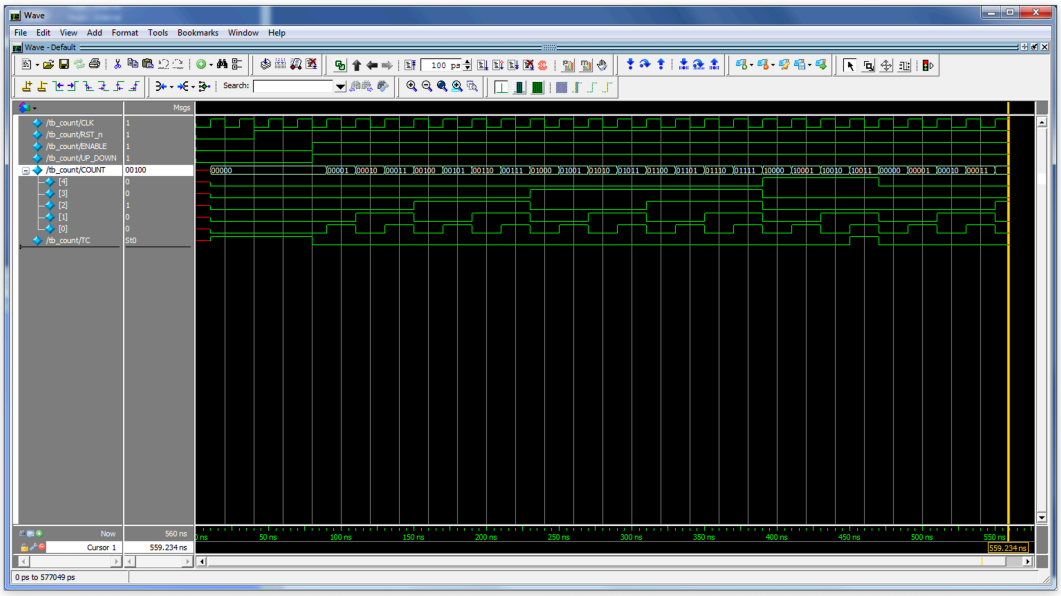
En el menú superior las lupas, permiten hacer zoom sobre los resultados de la simulación.

In the top menu the magnifying glasses allow you to zoom in on the results of the simulation.



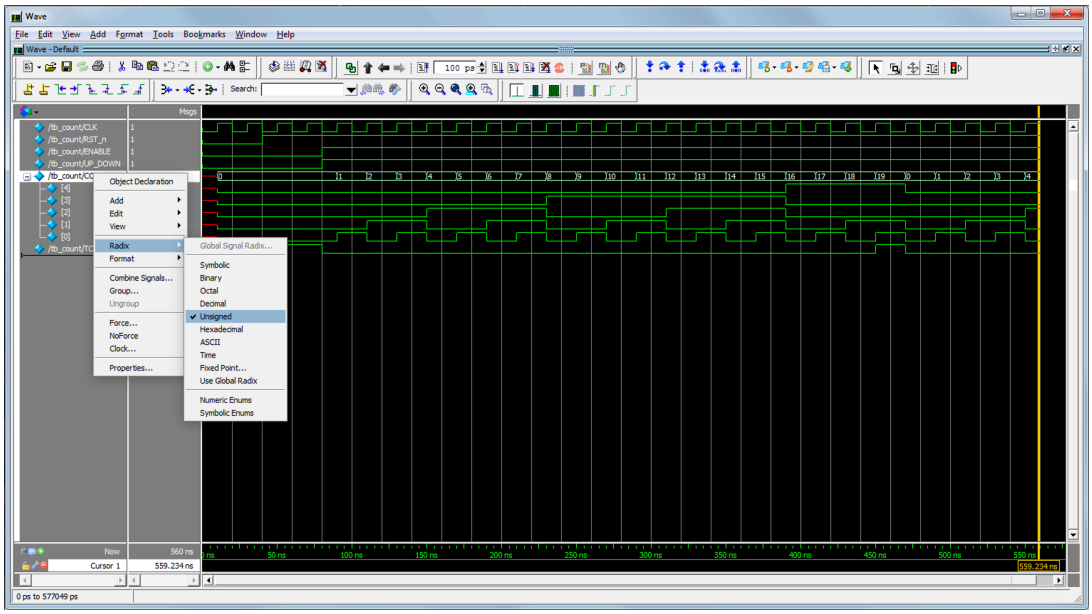
Inicialmente el ModelSim-Altera, ha realizado la simulación hasta que ha encontrado el `$stop` en el código Verilog-HDL del TestBench. Se visualizan todas las señales y se puede ver la cuenta en binario del contador.

Initially the ModelSim-Altera, has performed the simulation until it has found the `$stop` in Verilog-HDL code of TestBench. All the signals are displayed and the binary counter count can be seen.



Se puede cambiar la representación de las señales, en este caso se pueden visualizar el contador en formato decimal en lugar de binario, seleccionando la señal con el botón derecho del ratón y en la opción de **Radix** escogiendo **Unsigned**, para ver las cuentas en formato decimal sin signo.

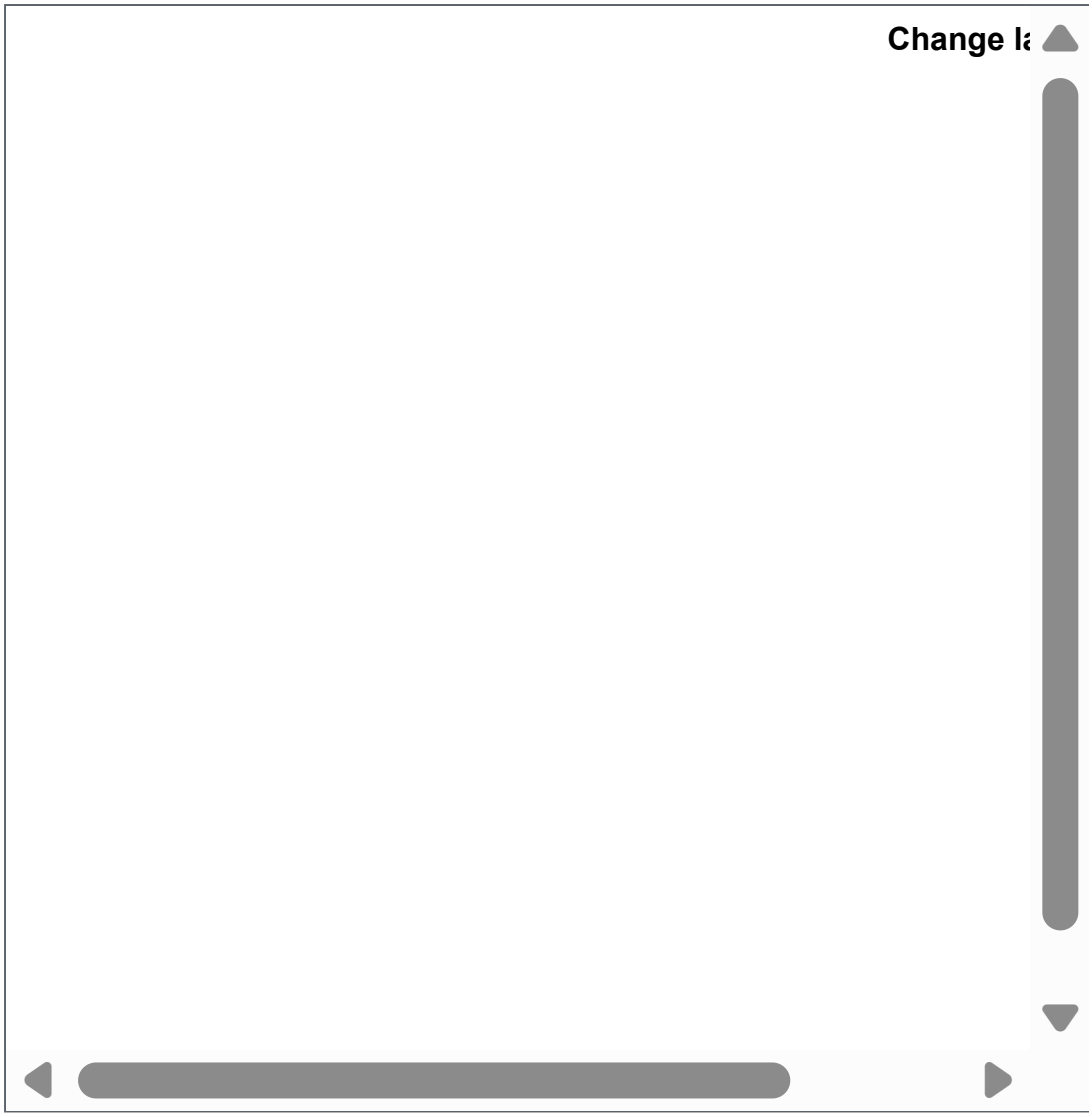
You can change the representation of the signals, in this case you can display the counter in decimal format instead of binary, selecting the signal with the right mouse button and in the option of **Radix** choosing **Unsigned**, to see the accounts in decimal format without sign.



En la barra de menú se encuentran los botones que permiten realizar la simulación, como se ve en la Figura inferior. De izquierda a derecha los botones tienen la siguiente funcionalidad: El primero sirve para realizar un reinicio de la simulación, es útil cuando se añaden más señales al visor de ondas y hay que volver a relanzar la simulación. A continuación se encuentra la casilla donde se establece el tiempo de la simulación, justo al lado está el botón que lanza la simulación durante el tiempo indicado. El último botón, el de stop, sirve para detener la simulación.

In the menu bar there are buttons that allow you to perform the simulation, as shown in the Figure below. From left to right the buttons have the following functionality: The first serves to perform a simulation restart, it is useful when more signals are added to the wave viewer and the simulation has to be re-launched. Next is the box where the simulation time is set, right next to it is the button that launches the simulation during the indicated time. The last button, the stop button, serves to stop the simulation.

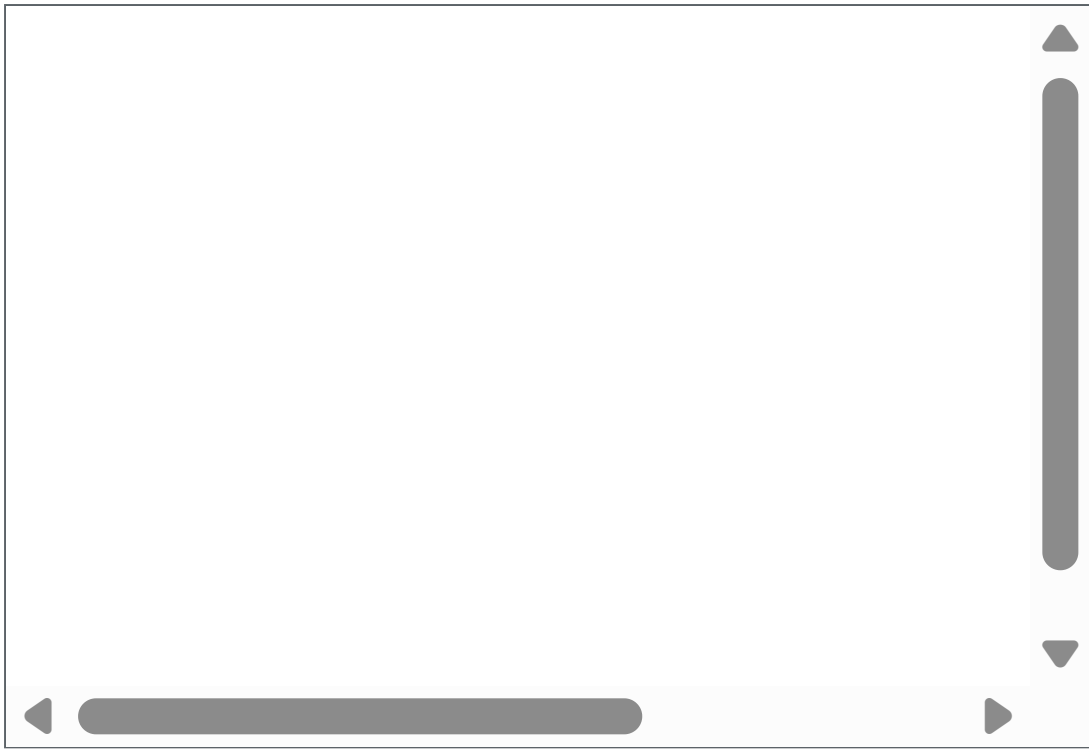
ABRIR EN UNA VENTANA NUEVA



Se puede volver a realizar la simulación pulsando en el botón de **Restart** y luego lanzar la simulación con **Run -All**. La simulación se detendrá en el `$stop` del código Verilog-HDL del Testbench. Estas operaciones también se pueden realizar escribiendo los comandos directamente en la ventana de mensajes. También se puede establecer un tiempo de simulación, por ejemplo de 300 ns y lanzar la simulación con **Run**. En este caso la simulación se detendrá a los 300 ns, ya que tienen lugar antes del `$stop`.

The simulation can be carried out again by clicking on the **Restart** button and then launching the simulation with **Run -All**. The simulation will stop at the `$stop` of the Verilog-HDL code of the Testbench. These operations can also be performed by typing the commands directly in the Message Window. You can also set a simulation time, for example 300 ns and launch the simulation with **Run**. In this case the simulation will stop at 300 ns, since they take place before the `$stop`.

ABRIR EN UNA VENTANA NUEVA

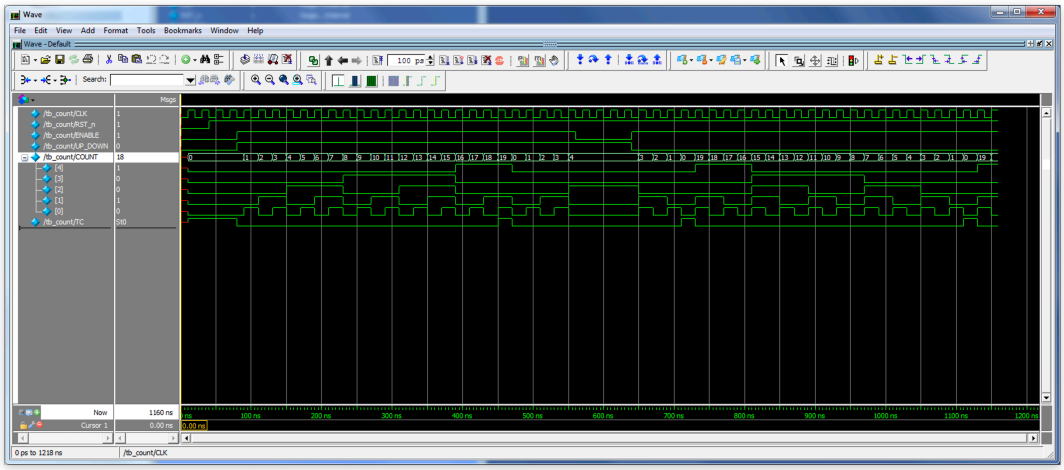


Comprobación de Resultados

Tras la primera simulación, se debe modificar el archivo de TestBench, para realizar una simulación completa del contador donde se puedan apreciar todos los casos de funcionamiento del mismo. Como sugerencia se pueden realizar valoraciones de los flancos del reloj para establecer los cambios de las señales. La siguiente imagen muestra un ejemplo de una simulación donde se aprecia como funciona el contador en modo ascendente o descendente. ¿Se han evaluado todos los casos posibles de funcionamiento del contador?

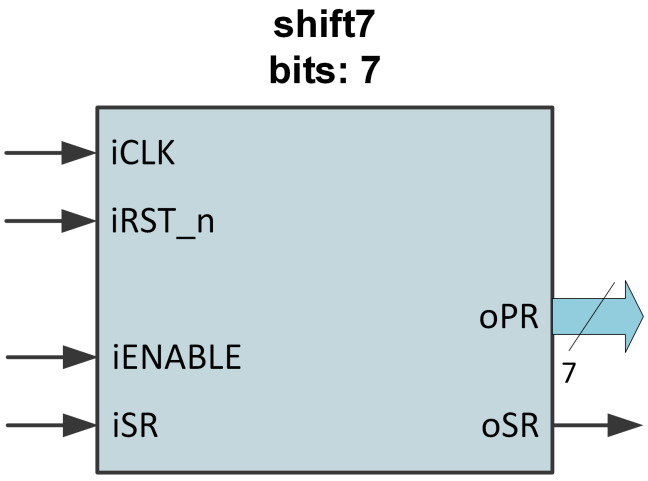
Checking Results

After the first simulation, the TestBench file must be modified to perform a complete simulation of the counter where all the operation cases of the meter can be appreciated. As a suggestion, it is possible to make valuations of the flanks of the clock to establish the changes of the signals. The following image shows an example of a simulation showing how the meter works in ascending or descending mode. Have all possible cases of meter operation been evaluated?



Subtarea 2: Diseño de un registro de desplazamiento

Subtask 2: Design a shift register



Realizar un registro de desplazamiento mediante descripción Verilog HDL, cuyas características sean las siguientes:

- Registro de desplazamiento de 7 bits síncrono.
- Reloj activo por flanco de subida.
- Línea de reset síncrona, activa a nivel bajo.
- Entrada serie.
- Salida serie.
- Salida paralelo.
- Entrada de habilitación de desplazamiento **enable** que permite el desplazamiento.
- Utilizar la Familia Cyclone IV E de intelFPGA.

Verifique su funcionamiento mediante una simulación que consideré todos los posibles casos de combinaciones de las señales de entrada y realice un barrido completo de las señales de salida.

Perform a shift register using Verilog HDL description, whose characteristics are the following:

- Synchronous 7-bit shift register.
- Active clock by rising edge.
- Synchronous reset line, active at low level.
- Serial input.
- Serial output.
- Parallel output.
- Enable input to shift.
- Use the Cyclone IV E family of intelFPGA.

Verify its operation through a simulation that considers all the possible cases of combinations of the input signals and perform a complete sweep of the output signals.

Subtarea 3: Diseño del juego de luces del coche fantástico

Subtask 3: Knight Rider light design

Realizar en los LEDs verdes LEDG7 a LEDG0 de la placa DE2-115 el juego de luces siguiente:

Carry out the following set of lights on the green LEDs (LEDG7 to LEDG0) of the DE2-115 board:

Ciclos	LEDG7	LEDG6	LEDG5	LEDG4	LEDG3	LEDG2	LEDG1	LEDG0
1	■							
2		■						
3			■					
4				■				
5					■			
6						■		
7							■	
8								■
9							■	
10						■		
11					■			
12				■				
13			■					
14		■						

En este diseño debe reutilizar el contador parametrizable de la Subtarea 1, y el registro de desplazamiento de la Subtarea 2, con las siguientes consideraciones:

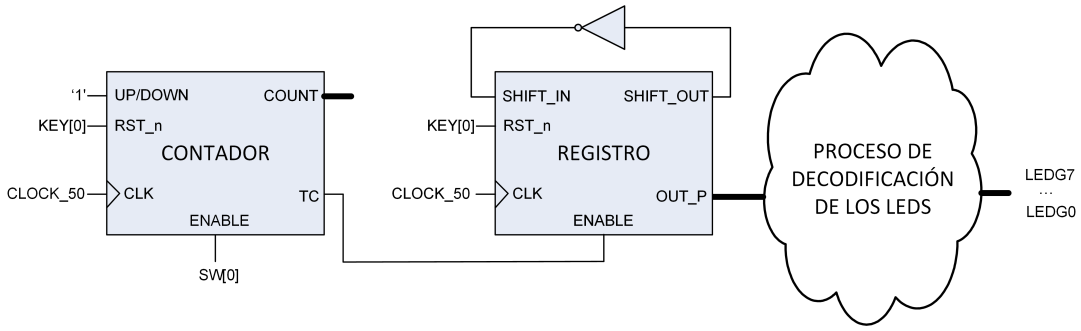
- El contador (que cuenta con la señal de reloj de la placa DE2 de 50 MHz) se empleará para habilitar con su salida TC la señal de enable del registro de desplazamiento, de forma que el desplazamiento sea cada 0,25 segundos.
- El registro de desplazamiento debe funcionar como un contador Johnson, por lo que generará 14 estados que corresponderán a los 14 ciclos del juego de luces.
- Además deberá crear un decodificador que a partir de los estados del contador Johnson genere la salida a los LEDs de LEDG7 a LEDG0.
- El enable del contador debe estar manejado por el estado del interruptor SW[0], de la placa DE2.

El top de la jerarquía del diseño debe ser un fichero Verilog HDL en donde se realice la instanciación del contador y del registro de desplazamiento, y el código asociado al proceso del decodificador.

In this design, you must reuse the parameterizable counter of Subtask 1, and the shift register of Subtask 2, with the following considerations:

- The counter (which has the clock signal of the 50 MHz DE2 board) will be used to enable with its TC output the enable signal of the shift register, so that the displacement is every 0.25 seconds.
- The shift register should work like a Johnson counter, so it will generate 14 states that correspond to the 14 cycles of the light set.
- In addition, you must create a decoder that, from the states of the Johnson counter, generates the output to the LEDs from LEDG7 to LEDG0.
- The counter enable must be controlled by the state of switch SW [0], of board DE2.

The top of the design hierarchy must be a Verilog HDL file where the instantiation of the counter and the shift register is performed, and the code associated with the decoder process.



Verificación con testbench

Verifica el funcionamiento del diseño mediante una simulación en ModelSim. Para ello tendrás que crear un testbench de manera similar a como lo has hecho en las subtareas 1 y 2.

NOTA: *Para que la simulación sea más rápida, es recomendable bajar el módulo del contador utilizado para temporizar las luces. Puede utilizarse, por ejemplo, un valor que haga que las luces se actualicen cada 0.25 ms en lugar de cada 0.25 s.*

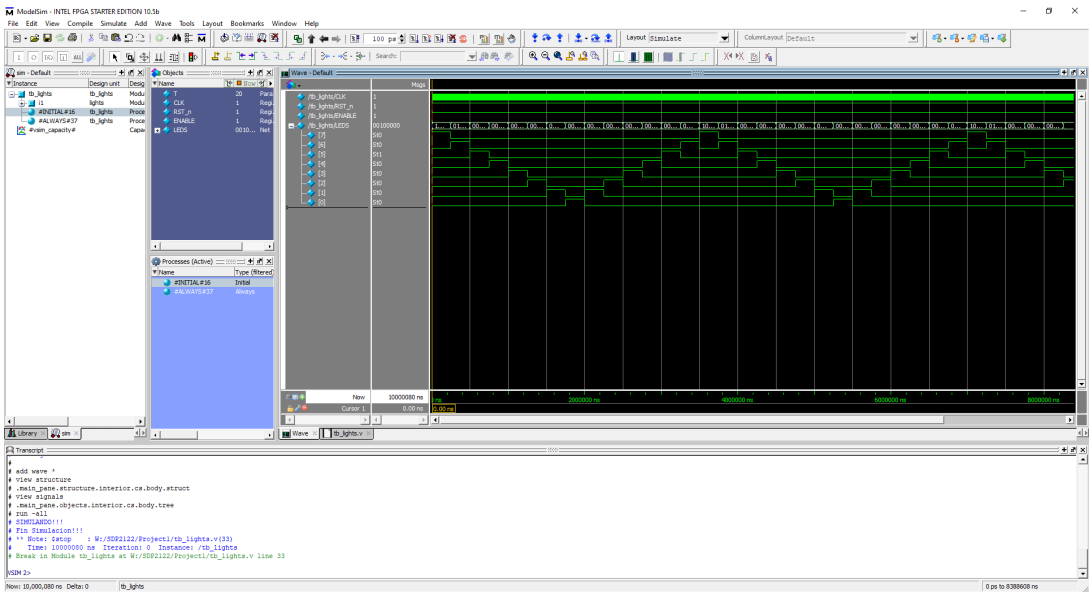
El resultado de la simulación debería ser similar al de la captura siguiente:

Verification with testbench

Verify the operation of your design with a ModelSim simulation. To accomplish this, you will need to create a testbench in a similar way as you have already done in subtasks 1 and 2.

NOTE: *In order to speed up the simulation, it is recommended to decrease the modulus of the counter used to time the lights. For instance, you may use a value such that the lights change every 0.25 ms instead of 0.25 s.*

The simulation result should look similar to this screenshot:

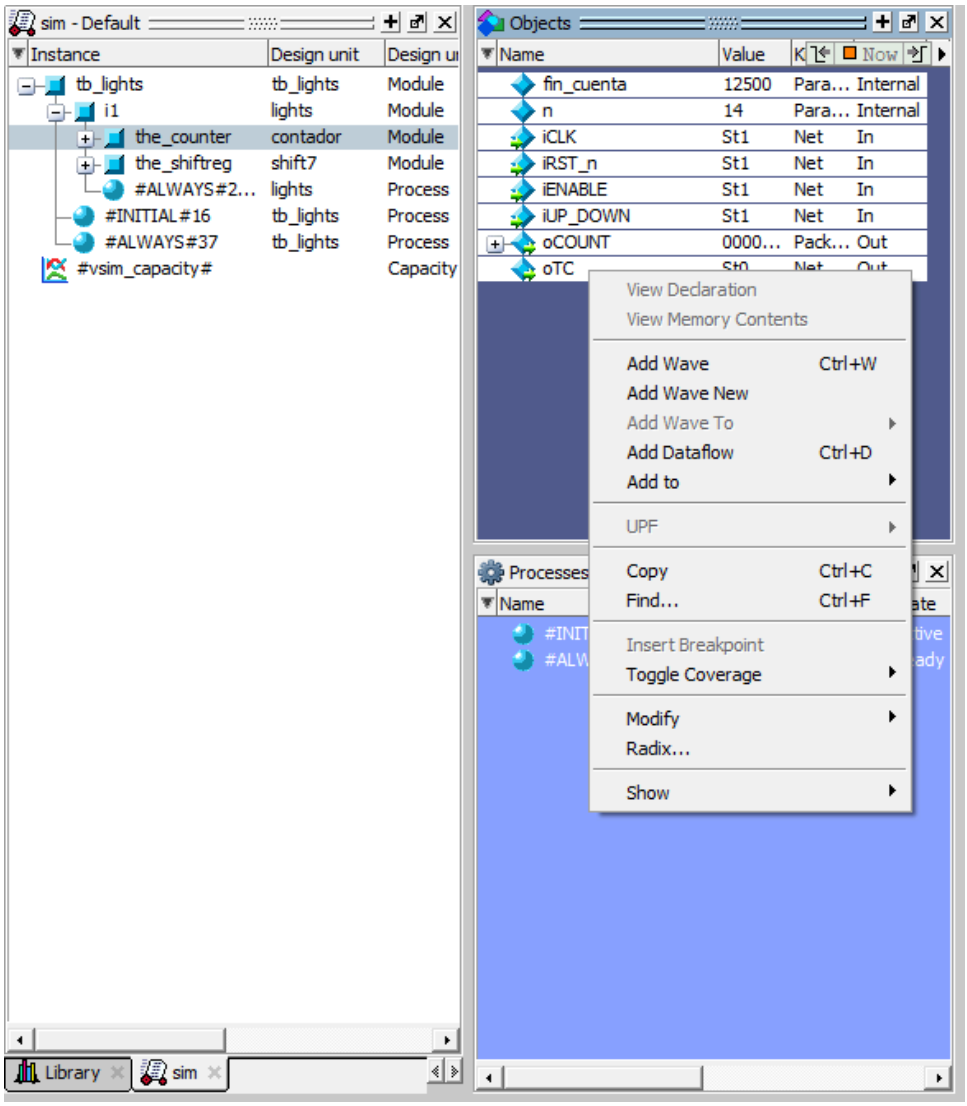

ModelSim - INTEL FPGA STARTER EDITION 10.5b
File Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help
sim - Default
Objects
Wave - Default
Processes (Active)
Transcript
add wave *
view structure
.main_pane.structure.interior.cs.body.struct
view signals
.main_pane.objects.interior.cs.body.tree
run -all
SIMULANDO!!!
Fin Simulacion!!!
** Note: $stop : W:/SDP2122/Proyecto1/tb_lights.v(33)
Time: 10000000 ns Iteration: 0 Instance: /tb_lights
Break in Module tb_lights at W:/SDP2122/Proyecto1/tb_lights.v line 33
VSIM 2>

El diseño que hemos realizado es jerárquico y contiene instancias de un contador y un registro de desplazamiento,. Sin embargo, tal vez hayas observado que en la simulación no estamos viendo las señales internas de estos submódulos, sino que solo vemos las señales presentes en el top del testbench. A menudo querremos visualizar también estas señales internas de los módulos instanciados en nuestro diseño, sobre todo si necesitamos depurarlo porque vemos que no funciona correctamente. Veamos ahora cómo hacerlo.

Para mostrar estas señales debemos utilizar las dos subventanas en la parte izquierda de la ventana de ModelSim: *Structure* y *Objects*. En la primera, a la izquierda del todo, podemos ver y desplegar una jerarquía completa de nuestro testbench y el diseño instanciado, en la que vemos el nombre de cada módulo instanciado y el nombre concreto que le hemos dado a cada instancia. Por ejemplo, en la imagen inferior, nuestro testbench **tb_lights** contiene una instancia de nuestro diseño **lights** a la que hemos llamado `i1`. El módulo **lights**, a su vez, contiene una instancia del contador **contador** a la que hemos llamado `the_counter`, y una instancia del registro **shift7** a la que hemos llamado `the_shiftreg`.

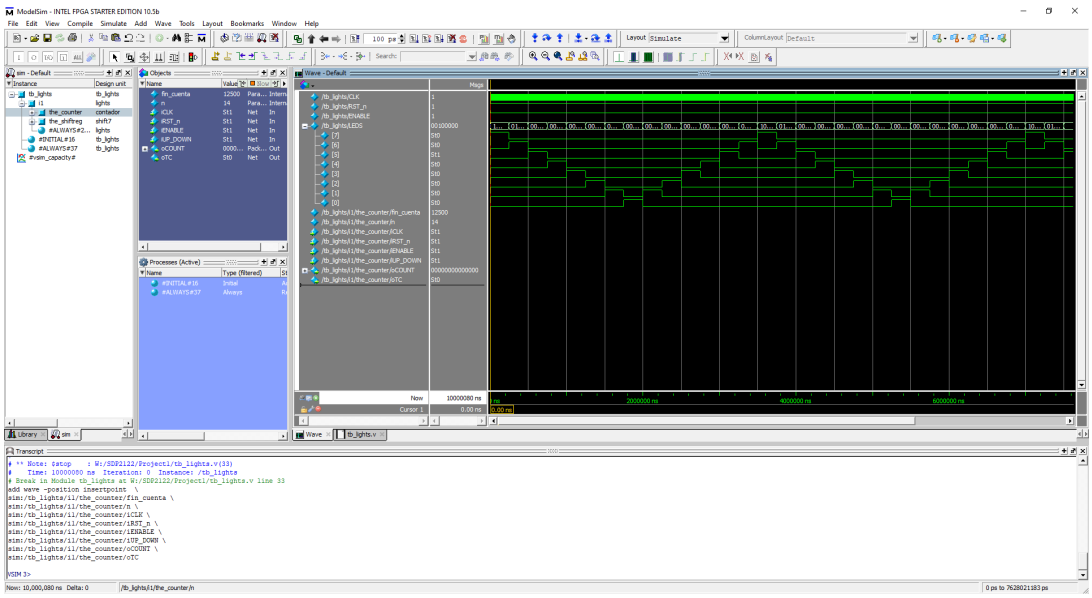
The circuit that we have designed is a hierarchical one, and contains instances of a counter and a shift register. However, you may have observed that the simulation is not displaying the internal signals in these submodules, we can only see those signals that are present in the testbench's top module. We will often want to display these internal signals of instanced modules as well, especially when we are trying to debug our design if it doesn't work correctly. Let us see how we can do that.

To display these signals we need to use the two subwindows located on the left part of the ModelSim window: *Structure* and *Objects*. In the first (leftmost) one, we may observe and unfold the complete hierarchy of our testbench and the instanced design, and we see the name of every instanced module and the particular name given to each instance. For instance, in the image below, our testbench tb_lights contains an instance of our design lights that we have called `i1`. In turn, the **lights** module contains an instance of the counter module **contador** that we have called`the_counter`, and an instance of the shift register module **shift7** that we have called `the_shiftreg`.



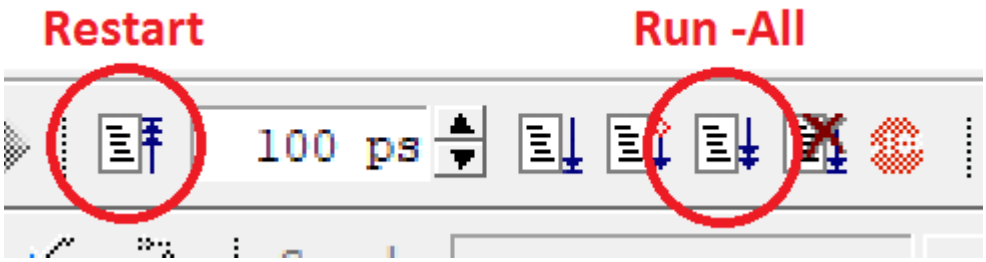
Cuando seleccionamos una instancia en la jerarquía, en la ventana de *Objects* aparece una lista de todas las señales definidas dentro de ese módulo, como se ve en la imagen superior. Podemos entonces seleccionar las que nos interesen, hacer click derecho y elegir la opción **Add Wave** para añadir todas las señales elegidas a la ventana de visualización de formas de onda (*Wave*), como se ve en la siguiente captura:

When we click on an instance in the hierarchy, the *Objects* window will show a list of all signals that are defined within that module, as we may see in the image above. We may then select all signals of interest, right click and select the command **Add Wave** in order to add all selected signals to the waveform display window (*Wave*), as in this screenshot:



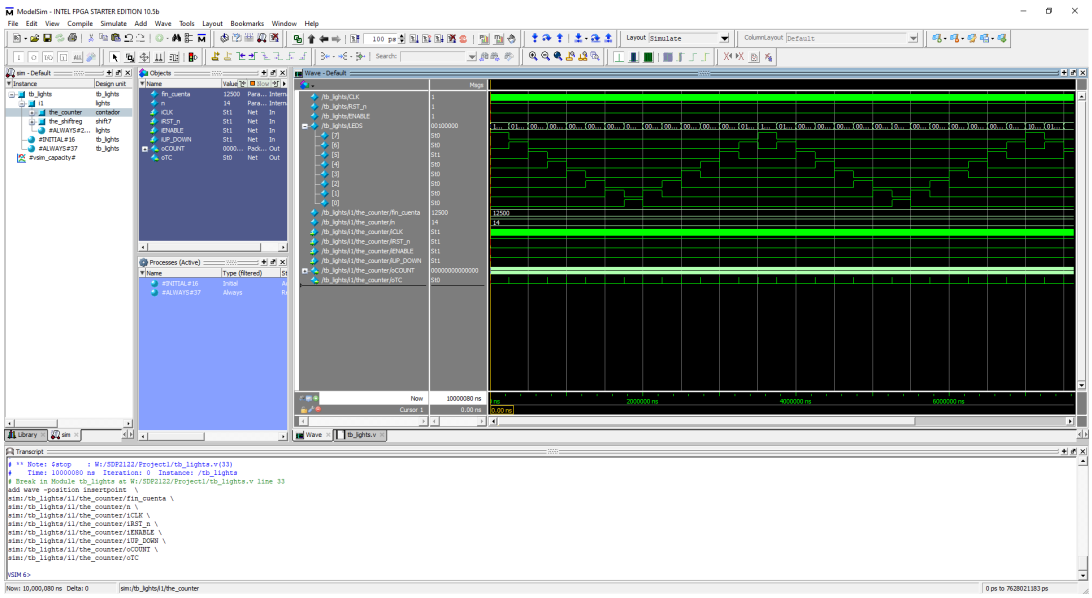
Observa que, aunque las señales aparecen ahora en la lista de la ventana *Wave*, no vemos todavía aún sus formas de onda. El motivo es el siguiente: las señales aún no estaban en esta ventana cuando hemos lanzado la simulación, y por ello ModelSim no se ha molestado en generarlas. Para poder ver su forma, tenemos relanzar la simulación desde dentro de ModelSim (¡sin cerrar la ventana!). Para ello podemos utilizar, en este orden, los botones **Restart** y **Run -All** que tenemos disponibles en las barras de iconos de ModelSim:

The selected signals now appear in the list on the *Wave* window, however we're still not able to see their waveforms. The reason is as follows: these signals weren't present yet on the *Wave* window when the simulation was launched, and so ModelSim didn't bother to generate them. In order to display their waveforms, we must launch the simulation again from within the ModelSim window (without closing it!). We may do that using the **Restart** and **Run -All** buttons (in this order) that are available on the ModelSim icon bars:



El comando **Restart** devuelve la simulación al instante de inicio (t = 0), y el comando **Run -All** vuelve a ejecutar la simulación hasta encontrar el comando de finalización `$stop` en el testbench. El resultado es el de la captura siguiente, donde por fin podemos ver las señales internas del contador:

The **Restart** command brings the simulation back to the starting point (t = 0), and the **Run -All** command executes the simulation again until it finds the stopping command `$stop` within the testbench. As a result we obtain the following screenshot, where we may finally see the counter's internal signals:



Realiza la misma operación para poder ver también las señales internas del registro de desplazamiento que has instanciado en tu diseño, y haz una captura de pantalla para incluirla en la entrega de la Tarea.

Repeat this operation in order to display also the internal signals of the shift register instanced in your design. Take a screenshot and include it in this Project's report when you submit it.

Verificación en placa

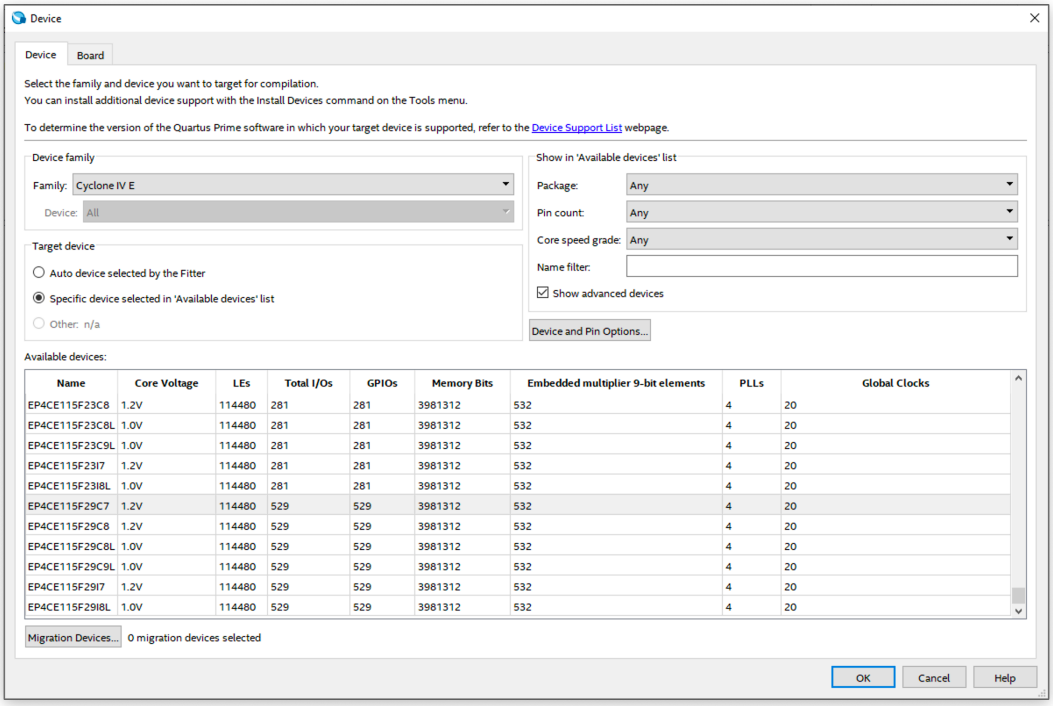
On-board verification

Verifica el funcionamiento del diseño en la placa DE2-115, para lo cual se tendrá que realizar la selección del dispositivo (FPGA) que hay en la placa DE2-115 y la asignación de pines correspondiente entre el diseño y la FPGA.

La selección del dispositivo se realiza en el menú **Assignments > Device...** de Quartus, ventana que se muestra en la siguiente Figura. Hay que escoger el dispositivo **EP4CE115F29C7** de la familia Cyclone IV E.

Verify the operation of your design on the DE2-115 board. To accomplish this, it will be necessary to select the device (FPGA) that is present on the DE2-115 board and the corresponding pin assignments between the design and the FPGA.

The device is selected in the **Assignments > Device...** menu of Quartus, which opens the window shown in the following Figure. The device **EP4CE115F29C7** of the Cyclone IV E family must be chosen.



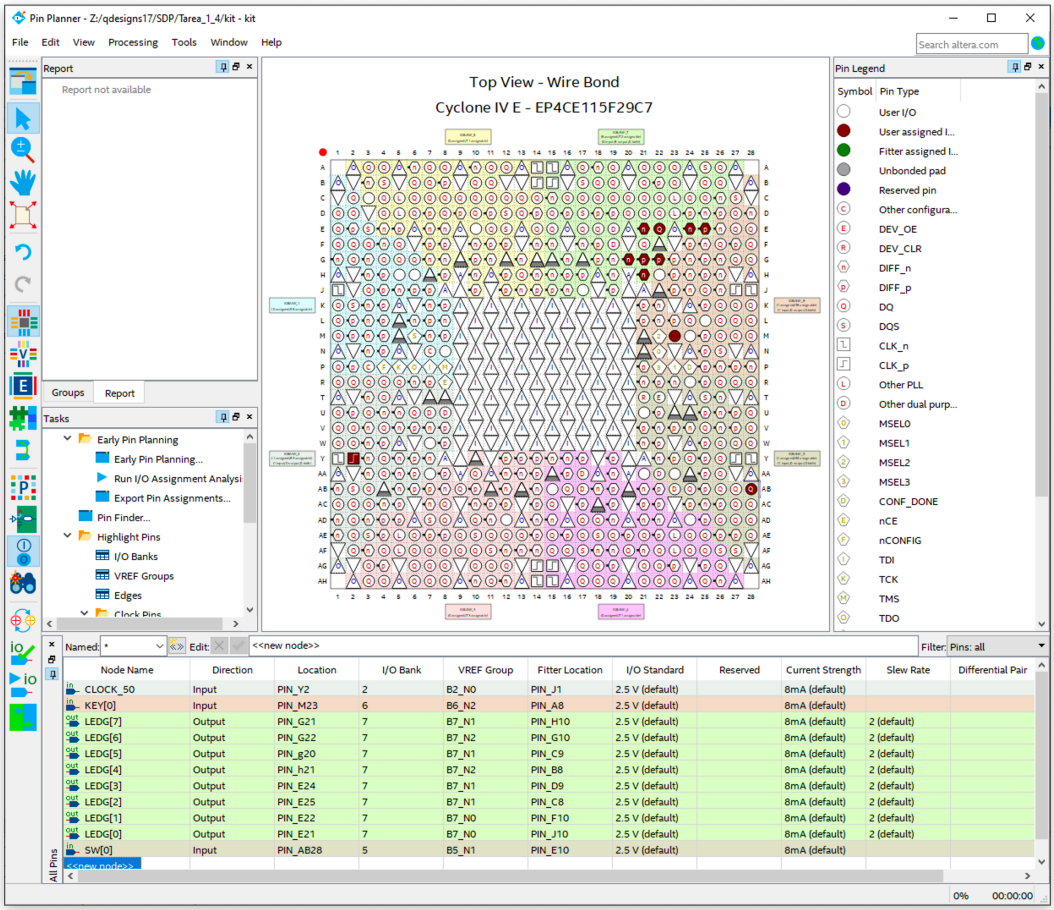
La asignación de pines consiste en indicar a que pin de la FPGA, debe conectarse cada una de las señales de entrada y de salida del diseño que se quiere verificar. Esto requiere que en primer lugar se identifiquen las señales de entrada y salida del diseño y a que pines de la FPGA deben de conectarse. En la siguiente tabla se muestra la información necesaria, para realizar la asignaciones de pines.

Pin assignment consists in indicating to which pin of the FPGA each of the input and output signals of the design to be verified must be connected. This requires first identifying the input and output signals of the design and which pins of the FPGA should be connected. The following table shows the necessary information to make the pin assignments.

Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA
CLOCK_50	Y2	LEDG[0]	E21
		LEDG[1]	E22
KEY[0]	M23	LEDG[2]	E25
KEY[1]	M21	LEDG[3]	E24
		LEDG[4]	H21
SW[0]	AB28	LEDG[5]	G20
SW[1]	AC28	LEDG[6]	G22
		LEDG[7]	G21

La asignación de pines se realiza en el menú **Assignments > Pin Planner** de Quartus, ventana que se muestra en la siguiente Figura. En la parte superior de la Figura se ve una vista inferior del encapsulado del dispositivo. Los 780 pines que tiene la Cyclone IV, se distribuyen en una matriz cuadrada en la que se enumeran las filas mediante letras y las columnas mediante números. En la parte inferior de la ventana mostrada en la Figura, se ve la lista de señales de entrada y salida del diseño, para cada señal se indica el nombre (**Node Name**), la dirección (**Direction**), la conexión (**Location**), y otros datos. Es en la columna de **Location** donde se debe especificar el pin al que se quiere conectar la señal. Por ejemplo, una señal de reloj del sistema (CLOCK_50) irá conectada al oscilador de la placa, según la Tabla superior el oscilador de 50 MHz se encuentra físicamente conectado al pin Y2 de la FPGA, por tanto en la columna **Location** de la señal se debe poner el valor Y2.

Pin assignments are configured in the menu **Assignments > Pin Planner** of Quartus, in the window shown in the following figure. In the upper part of the Figure you can see a bottom view of the device package. The Cyclone IV device has 780 pins, distributed in a square matrix in which rows are labeled by letters and columns by numbers. In the lower part of the window shown in the Figure, you can see the list of input and output signals of the design, for each signal we see the name (**Node Name**), the address (**Direction**), the connection (**Location**), and other data. In the **Location** column, you must specify the pin to which you want to connect the signal. For example, a system clock signal (CLOCK_50) will be connected to the oscillator of the board, according to the Table above the 50 MHz oscillator is physically connected to the Y2 pin of the FPGA, therefore in the Location column of the CLOCK_50 you must put the value PIN_Y2.


Pin Planner - Z:/qdesigns17/SDP/Tarea_1_4/kit - kit
File Edit View Processing Tools Window Help
Search altera.com
Report
Report not available
Groups Report
Tasks
Early Pin Planning
Early Pin Planning...
Run I/O Assignment Analysi
Export Pin Assignments...
Pin Finder...
Highlight Pins
I/O Banks
VREF Groups
Edges
Top View - Wire Bond
Cyclone IV E - EP4CE115F29C7
Pin Legend
Symbol Pin Type
User I/O
User assigned I...
Fitter assigned I...
Unbonded pad
Reserved pin
Other configura...
DEV_OE
DEV_CLR
DIFF_n
DIFF_p
DQ
DQS
CLK_n
CLK_p
Other PLL
Other dual purp...
MSEL0
MSEL1
MSEL2
MSEL3
CONF_DONE
nCE
nCONFIG
TDI
TCK
TMS
TDO
Named: *
Edit: <<new node>>
Filter: Pins: all
Node Name Direction Location I/O Bank VREF Group Fitter Location I/O Standard Reserved Current Strength Slew Rate Differential Pair
CLOCK_50 Input PIN_Y2 2 B2_N0 PIN_J1 2.5 V (default) 8mA (default)
KEY[0] Input PIN_M23 6 B6_N2 PIN_A8 2.5 V (default) 8mA (default)
LEDG[7] Output PIN_G21 7 B7_N1 PIN_H10 2.5 V (default) 8mA (default) 2 (default)
LEDG[6] Output PIN_G22 7 B7_N2 PIN_G10 2.5 V (default) 8mA (default) 2 (default)
LEDG[5] Output PIN_g20 7 B7_N1 PIN_C9 2.5 V (default) 8mA (default) 2 (default)
LEDG[4] Output PIN_h21 7 B7_N2 PIN_B8 2.5 V (default) 8mA (default) 2 (default)
LEDG[3] Output PIN_E24 7 B7_N1 PIN_D9 2.5 V (default) 8mA (default) 2 (default)
LEDG[2] Output PIN_E25 7 B7_N1 PIN_C8 2.5 V (default) 8mA (default) 2 (default)
LEDG[1] Output PIN_E22 7 B7_N0 PIN_F10 2.5 V (default) 8mA (default) 2 (default)
LEDG[0] Output PIN_E21 7 B7_N0 PIN_J10 2.5 V (default) 8mA (default) 2 (default)
SW[0] Input PIN_AB28 5 B5_N1 PIN_E10 2.5 V (default) 8mA (default)
<<new node>>
All Pins
0% 00:00:00

Una vez hecha la asignación de pines, se cierra la ventana y se vuelve a compilar el diseño, en el menú: **Processing > Start Compilation**. La nueva compilación tendrá en cuenta a que pin debe conectar cada entrada y salida del diseño.

La verificación final del diseño en la placa DE2-115, se realiza mediante la configuración de la Cyclone IV en modo JTAG. Conectar la alimentación de la placa DE2-115 (¡asegurarse de que está enchufada!). En el menú de Quartus seleccionar: **Tools > Programmer**, para que aparezca la ventana del programador, mostrada más abajo.

Si el modo **JTAG** no aparece seleccionado en el recuadro **Mode**, habrá que seleccionarlo. Si por defecto no aparece seleccionado el USB-Blaster, hay que pulsar en **Hardware Setup** y seleccionar el USB-Blaster. Debe aparecer en la lista de la ventana un fichero de configuración `.sof`, con el mismo nombre que el proyecto de Quartus. Si esto no ocurriese hay que añadirlo mediante el botón **Add**. La casilla **Program/Configure** debe ser marcada.

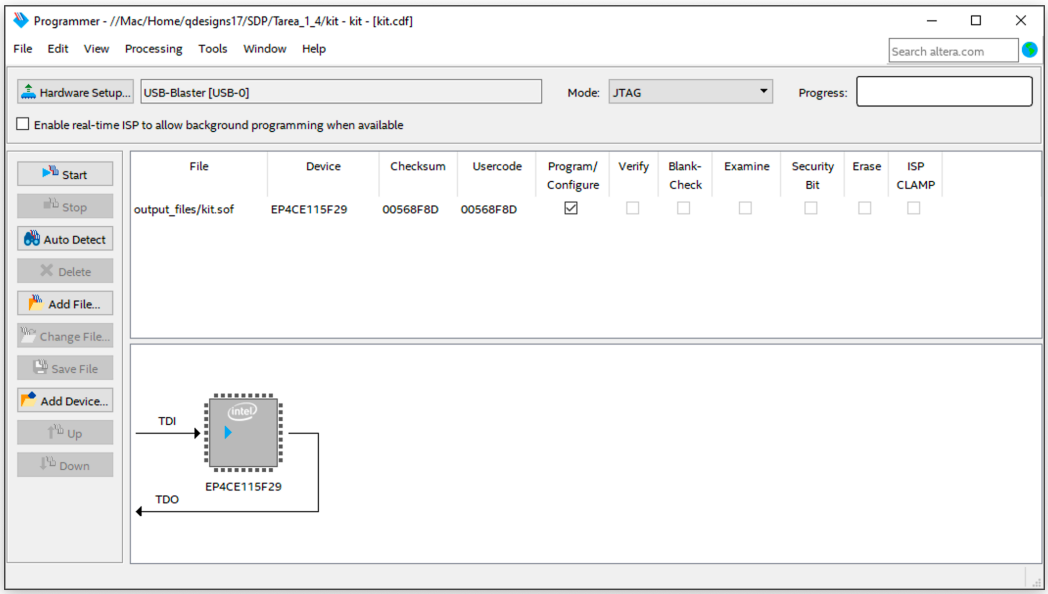
En este punto la ventana del programador debe aparecer tal como se muestra en la imagen inferior. Solo hay que pulsar el botón **Start** para realizar la configuración de la FPGA. Verifica el funcionamiento del juego de luces en la placa DE2-115.

Once pin assignment is done, close the window and recompile the design using the menu option **Processing > Start Compilation**. The new compilation will take into account the pin connections of each input and output of the design.

The final design verification on the DE2-115 board is done by configuring the Cyclone IV in JTAG mode. Turn on the power of the DE2-115 board (make sure it is plugged in!). In the Quartus menu select: **Tools > Programmer**, to display the programmer window, shown below.

Select the JTAG mode in the **Mode** box if it is not selected already. If the USB-Blaster is not selected by default, click on **Hardware Setup** and select the USB-Blaster. A `.sof` configuration file with the same name as your Quartus project should appear in the window list. If this does not happen, it must be added using the **Add** button. The **Program/Configure** box must be checked.

At this point the programmer window should appear as shown in the image below. Just press the Start button to configure the FPGA. Check the operation of the light set on the DE2-115 board.


Programmer - //Mac/Home/qdesigns17/SDP/Tarea_1_4/kit - kit - [kit.cdf]
File Edit View Processing Tools Window Help
Search altera.com
Hardware Setup...
USB-Blaster [USB-0]
Mode: JTAG
Progress:
Enable real-time ISP to allow background programming when available
Start
Stop
Auto Detect
Delete
Add File...
Change File...
Save File
Add Device...
Up
Down
File
Device
Checksum
Usercode
Program/ Configure
Verify
Blank-Check
Examine
Security Bit
Erase
ISP CLAMP
output_files/kit.sof
EP4CE115F29
00568F8D
00568F8D
TDI
TDO
EP4CE115F29